微机接口技术教程

Antigravity 辅助制作

2026 年 1 月 12 日

目录

1 微机系统基础 **2**

1.1 冯 • 诺依曼体系结构 2

1.2 存储程序工作原理 2

2 数据表示与运算 **2**

2.1 原码、反码与补码 2

2.2 BCD 码与 ASCII 码 2

3 8086 CPU 架构 **3**

3.1 CPU 基本功能 3

3.2 寄存器组 3

4 存储器组织 **3**

4.1 分段机制 3

4.2 字单元存储 3

5 指令系统与汇编语言 **3**

5.1 数据传送指令 (Data Transfer) 3

5.2 算术运算指令 (Arithmetic) 4

5.3 逻辑与移位指令 (Logical and Shift) 6

5.4 串操作指令 (String Operations) 7

5.5 控制转移指令 (Control Transfer) 8

5.6 子程序与中断 (Subroutines and Interrupts) 8

5.7 常用伪指令 9

5.8 DOS 功能调用 (INT 21H) 9

6 汇编语言程序结构 **10**

7 存储器硬件基础 **10**

7.1 存储器分类 10

7.2 地址译码方式 11

8 微机接口芯片（待续） **11**

1 微机系统基础

1.1 冯·诺依曼体系结构

冯·诺依曼体系结构是现代计算机的核心思想，其主要特点包括：

1. **采用二进制**：计算机内部采用二进制表示指令和数据。
2. **存储程序工作方式**：程序和数据一样存储在存储器中，由计算机自动执行。
3. **硬件五大部件**：输入设备、输出设备、存储器、运算器、控制器。

1.2 存储程序工作原理

- **编程序**：用户根据任务逻辑编写指令序列。
- **存储**：程序预先加载到存储器中。
- **自动执行**：控制器自动、连续地从存储器中取出指令并执行。

2 数据表示与运算

2.1 原码、反码与补码

1. **原码**：最高位为符号位（0 正 1 负），其余位表示数值绝对值。存在 ±0 不唯一问题。
2. **反码**：正数同原码；负数符号位保持，数值位按位取反。
3. **补码**：正数同原码；负数为反码加 1。
    - **优点**：零唯一；加减法统一；硬件实现简单。
    - **范围**（8 位）：$-128 \sim +127$。

2.2 BCD 码与 ASCII 码

- **BCD 码**：用 4 位二进制表示 1 位十进制数。分为压缩 BCD（一字节两位）和非压缩 BCD（一字节一位）。
- **ASCII 码**：美国标准信息交换代码。常用值：'0' (30H), 'A' (41H), 'a' (61H)。

3 8086 CPU 架构

3.1 CPU 基本功能

1. **指令控制**：控制指令执行顺序。
2. **操作控制**：产生操作控制信号。
3. **时间控制**：对操作进行定时。
4. **数据加工**：算术与逻辑运算。

3.2 寄存器组

- **通用寄存器**：AX (累加器), BX (基址), CX (计数), DX (数据)。
- **指针与变址**：SP (堆栈指针), BP (基址指针), SI (源变址), DI (目的变址)。
- **段寄存器**：CS (代码段), DS (数据段), SS (堆栈段), ES (附加段)。
- **指令指针**：IP。
- **标志寄存器**：FLAGS (含 CF, ZF, SF, OF, IF, DF 等)。

4 存储器组织

4.1 分段机制

8086 具有 20 位地址总线，可寻址 1MB 空间。由于寄存器为 16 位，采用分段机制：

$$\text{物理地址} = \text{段地址} \times 16 + \text{偏移地址} \tag{1}$$

4.2 字单元存储

采用小端格式（Little-endian）：低位字节存入低地址，高位字节存入高地址。

5 指令系统与汇编语言

5.1 数据传送指令 (Data Transfer)

1. **MOV (Move)**

- **功能**: 将源操作数传送到目的操作数。
- **使用场景**: 寄存器初始化、内存数据加载、通用数据交换。
- **影响**: 不影响任何标志位。可以改变通用寄存器、段寄存器或存储单元。

2. **XCHG (Exchange)**

- **功能**: 交换两个操作数的内容。
- **使用场景**: 在没有中间寄存器的情况下互换两个值。
- **影响**: 不影响标志位。不支持内存与内存之间的交换。

3. **LEA (Load Effective Address)**

- **功能**: 将源操作数的偏移地址送入目的寄存器。
- **使用场景**: 获取变量地址、复杂的地址计算。
- **影响**: 不影响标志位。常用格式 `LEA BX, OFFSET VAR`。

4. **PUSH/POP (Push/Pop onto Stack)**

- **功能**: 将数据压入堆栈或从堆栈弹出数据。
- **使用场景**: 子程序参数跳转、保存寄存器现场。
- **影响**: 不影响标志位（POP FLAGS 除外）。修改 SP 指针。

5. **IN/OUT (Input/Output)**

- **功能**: 在累加器（AL/AX）与 I/O 端口之间传输数据。
- **使用场景**: 与外部硬件通信。
- **影响**: 不影响标志位。只能使用 AL/AX 作为数据载体。

5.2 算术运算指令 (Arithmetic)

1. **ADD/ADC (Add / Add with Carry)**

- **功能**: 执行二进制加法（ADC 包含进位 CF）。
- **使用场景**: 普通加法计算、多字节/高精度加法链。
- **影响**: 影响 CF, ZF, SF, OF, AF, PF。

2. **SUB/SBB (Subtract / Subtract with Borrow)**

- **功能**: 执行二进制减法（SBB 包含借位 CF）。

- **使用场景**: 数值相减、多字节减法链。
- **影响**: 影响 CF, ZF, SF, OF, AF, PF。

3. **MUL/IMUL (Multiply / Integer Multiply)**
   - **功能**: 无符号/有符号乘法。
   - **使用场景**: 整数扩大倍数、数值运算。
   - **影响**: 影响 CF, OF（若高位有意义则置 1），其余标志位不确定。
4. **DIV/IDIV (Divide / Integer Divide)**
   - **功能**: 无符号/有符号除法。
   - **使用场景**: 整数均摊、取余运算。
   - **影响**: 标志位不确定。若除数为 0 或溢出会触发中断。
5. **INC/DEC (Increment/Decrement)**
   - **功能**: 操作数加 1 或减 1。
   - **使用场景**: 循环计数器更新、指针位移。
   - **影响**: 影响 ZF, SF, OF, AF, PF，**不影响 CF**。
6. **NEG (Negate)**
   - **功能**: 执行 0 减去操作数，即求二进补码的负数。
   - **使用场景**: 正负号反转。
   - **影响**: 只要操作数不为 0，CF 总是置 1。
7. **CBW/CWD (Convert Byte to Word / Word to Doubleword)**
   - **功能**: 将 AL 的符号位扩展到 AH，或将 AX 的符号位扩展到 DX。
   - **使用场景**: 有符号除法前的准备工作（防止数据溢出或符号丢失）。
   - **影响**: **不影响标志位**。改变 AH 或 DX 寄存器。
8. **BCD 调整指令 (BCD Adjustments)**
   - **功能**: 在执行二进制加减后，将结果调整为 8421 BCD 格式。
   - **使用场景**: 财务计算、BCD 码格式的数据运算。
   - **主要指令**: `DAA/DAS` (加/减后的十进制调整)、`AAA/AAS` (加/减后的 ASCII 调整)。
   - **影响**: 影响 CF, AF, ZF, SF, PF。

5.3 逻辑与移位指令 (Logical and Shift)

1. **AND (Logical AND)**
   - **功能**: 对两个操作数执行按位与运算。
   - **使用场景**: 屏蔽特定位（将不需要的位清零）。
   - **影响**: CF=0, OF=0; 影响 ZF, SF, PF。
2. **OR (Logical OR)**
   - **功能**: 对两个操作数执行按位或运算。
   - **使用场景**: 设置特定位（将需要的位置 1）。
   - **影响**: CF=0, OF=0; 影响 ZF, SF, PF。
3. **XOR (Exclusive OR)**
   - **功能**: 对两个操作数执行按位异或运算。
   - **使用场景**: 寄存器快速清零（如 `XOR AX, AX`）、位反转。
   - **影响**: CF=0, OF=0; 影响 ZF, SF, PF。
4. **NOT (Logical NOT)**
   - **功能**: 对操作数执行按位取反（一补码）。
   - **使用场景**: 位级逻辑取反。
   - **影响**: **不影响任何标志位**。
5. **TEST (Test Bits)**
   - **功能**: 执行按位与运算但不保存结果，仅更新标志位。
   - **使用场景**: 检测特定位是否为 1（如检测状态位）。
   - **影响**: CF=0, OF=0; 影响 ZF, SF, PF。
6. **SHL/SHR (Shift Logical Left/Right)**
   - **功能**: 逻辑左移/右移。
   - **使用场景**: 无符号数快速乘除 2 的幂次、位级拼接。
   - **影响**: 移出的位进入 CF。影响 ZF, SF, PF, OF。
7. **SAR (Shift Arithmetic Right)**

- **功能**: 算术右移（保持符号位）。
- **使用场景**: 有符号数快速除法。
- **影响**: 移出的位进入 CF。影响 ZF, SF, PF, OF。

8. **ROL/ROR/RCL/RCR (Rotate)**
   - **功能**: 循环移位（带或不带进位 CF）。
   - **使用场景**: 位模式循环、加密算法、位重组。
   - **影响**: 影响 CF 和 OF。

5.4 串操作指令 (String Operations)

1. **REP (Repeat Prefix)**
   - **功能**: 根据 CX 的值重复执行串指令。
   - **使用场景**: 批量数据移动、内存填充。
   - **影响**: 自动递减 CX。
2. **MOVSB/MOVSW (Move String Byte/Word)**
   - **功能**: 将源串 (SI) 数据传送到目的串 (DI)。
   - **使用场景**: 内存块大范围复制。
   - **影响**: 自动更新 SI 和 DI（方向受 DF 标志控制）。
3. **STOSB/STOSW (Store String Byte/Word)**
   - **功能**: 将 AL/AX 内容存入目的地 (DI)。
   - **使用场景**: 内存缓冲区快速清零或初始化。
   - **影响**: 自动更新 DI。
4. **LODSB/LODSW (Load String Byte/Word)**
   - **功能**: 从源地址 (SI) 加载数据到 AL/AX。
   - **使用场景**: 遍历字符串进行逐字符分析。
   - **影响**: 自动更新 SI。
5. **CMPSB/CMPSW (Compare Strings)**
   - **功能**: 比较源串 (SI) 与目的串 (DI) 的内容。

- **使用场景**: 字符串相等性检测。
- **影响**: 影响所有状态标志位，特别是 ZF（用于判断是否相等）。

6. **SCASB/SCASW (Scan String)**

- **功能**: 将 AL/AX 内容与目的串 (DI) 内容进行比较。
- **使用场景**: 在字符串中搜索特定字符或零结束符。
- **影响**: 影响所有状态标志位。

5.5 控制转移指令 (Control Transfer)

1. **JMP (Jump)**

- **功能**: 无条件改变程序执行顺序。
- **使用场景**: 循环跳转、分支结束后的汇合、程序入口跳转。
- **影响**: **不影响标志位**。修改 CS (若是段间) 和 IP。

2. **Jcc (Jump on Condition)**

- **功能**: 根据状态标志位的状态（如 ZF, CF, SF 等）决定是否跳转。
- **使用场景**: 实现 `if-else` 结构、循环终止条件。
- **常见指令**: `JZ/JE` (结果为 0/相等), `JNZ/JNE` (不为 0/不等), `JB/JA` (低于/高于 - 无符号), `JL/JG` (低于/高于 - 有符号)。
- **影响**: 不影响标志位。

3. **LOOP/LOOPE/LOOPNE (Loop Control)**

- **功能**: 以 CX 为计数器执行循环转移。
- **使用场景**: 定次数的重复代码执行。
- **影响**: **不影响标志位**。自动执行 $(CX) \leftarrow (CX) - 1$，且当 $CX \neq 0$ (及满足额外条件) 时跳转。

5.6 子程序与中断 (Subroutines and Interrupts)

1. **CALL (Call Subroutine)**

- **功能**: 调用过程（子程序）。将返回地址压栈并跳转到目标。
- **使用场景**: 函数封装、模块化程序设计。

- **影响**: 不影响标志位。修改 SP 并更新 IP/CS。

2. **RET (Return from Subroutine)**
   - **功能**: 从子程序返回。从堆栈弹出返回地址到 IP/CS。
   - **使用场景**: 结束函数执行。
   - **影响**: 不影响标志位。
3. **INT n (Interrupt)**
   - **功能**: 触发类型编号为 n 的软件中断。
   - **使用场景**: DOS 系统功能调用 (`INT 21H`)、BIOS 屏幕服务 (`INT 10H`)。
   - **影响**: 将 FLAGS, CS, IP 压栈。清除 IF 和 TF 标志（禁止后续中断）。
4. **IRET (Interrupt Return)**
   - **功能**: 从中断服务程序返回。从堆栈弹出 IP, CS, FLAGS。
   - **使用场景**: 结束中断处理，恢复原程序状态。
   - **影响**: 恢复 FLAGS 寄存器的原始值。

5.7 常用伪指令

- **数据定义：** `DB` (字节), `DW` (字), `DD` (双字), `DUP` (重复)。
- **符号定义：** `EQU` 或 `=` 用于定义常量。
- **段定义：** `SEGMENT` 和 `ENDS` 用于界定逻辑段。
- **段关联：** `ASSUME` 告诉汇编器段寄存器与逻辑段的对应关系。
- **地址操作符：** `OFFSET` 获取偏移地址，`SEG` 获取段地址。

5.8 DOS 功能调用 (INT 21H)

- **1 号：** 单字符输入（带回显），结果保存在 AL。
- **2 号：** 单字符输出，由 DL 提供字符。
- **9 号：** 输出字符串，DX 指向以 '$' 结尾的字符串首地址。
- **0AH：** 字符串输入，DX 指向缓冲区。
- **4CH：** 结束程序并返回 DOS。

6 汇编语言程序结构

标准 8086 汇编程序通常由数据段、堆栈段和代码段组成：

```
DATA SEGMENT
    MSG DB 'Hello,␣World!$'
DATA ENDS

CODE SEGMENT
    ASSUME CS:CODE, DS:DATA
START:
    MOV AX, DATA
    MOV DS, AX          ; 初始化 DS
    LEA DX, MSG
    MOV AH, 09H
    INT 21H             ; 显示字符串
    MOV AH, 4CH
    INT 21H             ; 返回 DOS
CODE ENDS
END START
```

7 存储器硬件基础

7.1 存储器分类

1. 按位置分类：

   - **内存**：CPU 直接访问，速度快，易失性（断电丢失）。
   - **外存**：通过 I/O 接口访问，容量大，持久化存储。
   - **Cache**：高速缓冲存储器，协调 CPU 与内存的速度差异。

2. 按物理特性分类：

   - **RAM**：随机存取存储器。SRAM（快，无需刷新，贵）和 DRAM（慢，需刷新，廉价）。
   - **ROM**：只读存储器。数据固化，断电不丢失。

7.2 地址译码方式

- **单译码**：一条选择线对应一个存储单元，结构简单，适合小容量。
- **双译码**：采用行、列地址线交叉选中单元，大幅减少译码线，适合大容量。

8 微机接口芯片（待续）